



University of
Nottingham

UK | CHINA | MALAYSIA

A large, high-resolution image of the Earth as seen from space, showing the Western Hemisphere with North and South America. The Earth is framed by a thin white rectangular border. The background is a deep black space filled with numerous small, distant stars.

COMP-2024 / COMP-2039
**Artificial Intelligence
Methods**

Lecture 9
**Advanced Search Techniques &
Applications**



Learning Outcomes

IDENTIFY

1. Policy Matrix Evolution for Generation of Heuristics
2. Data Science (Machine Learning)
Improved Hyper-heuristic Optimisation



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Policy Matrix Evolution for Generation of Heuristics

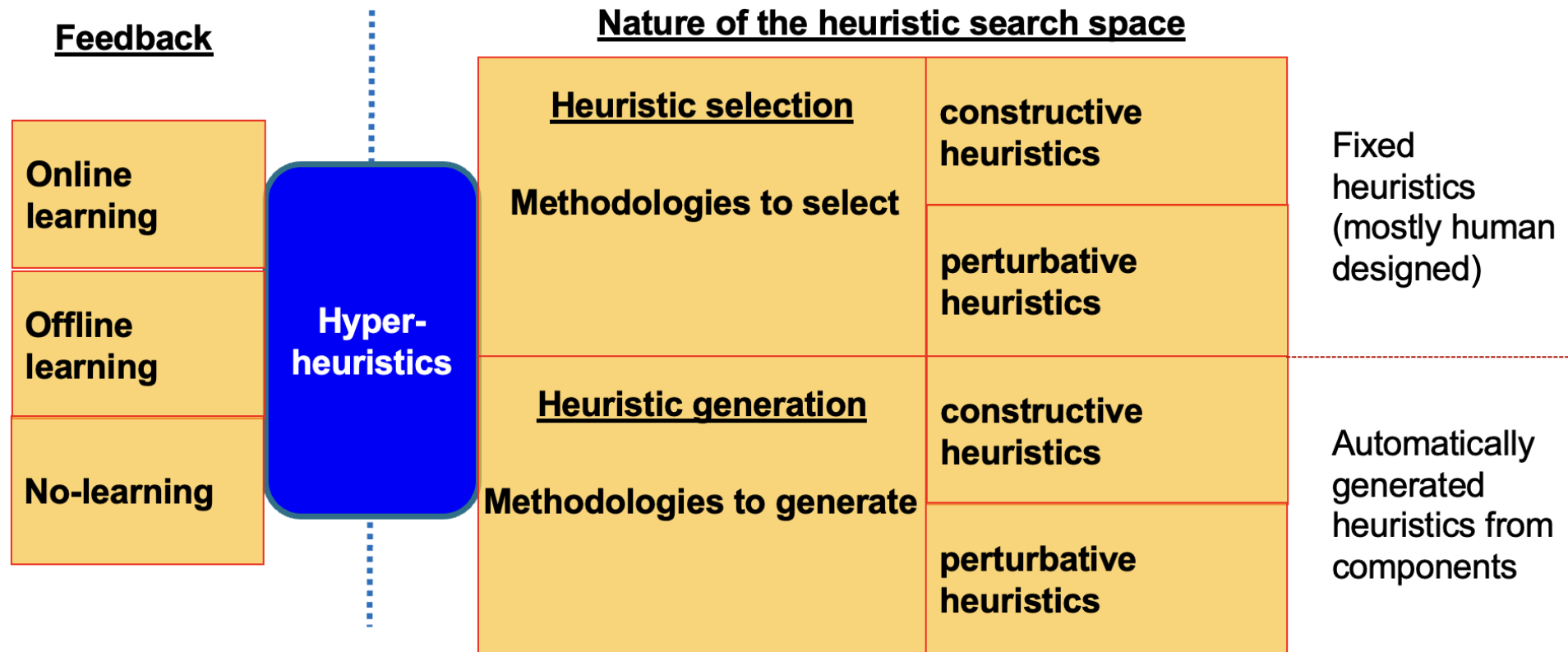
S. Asta, E. Özcan, and A.J. Parkes, CHAMP: Creating Heuristics via Many Parameters for Online Bin Packing, Expert Systems With Applications, vol. 63, pp. 208-221, doi:10.1016/j.eswa.2016.07.005, 2016

<https://www.sciencedirect.com/science/article/pii/S0957417416303499>



Remember **THIS...**

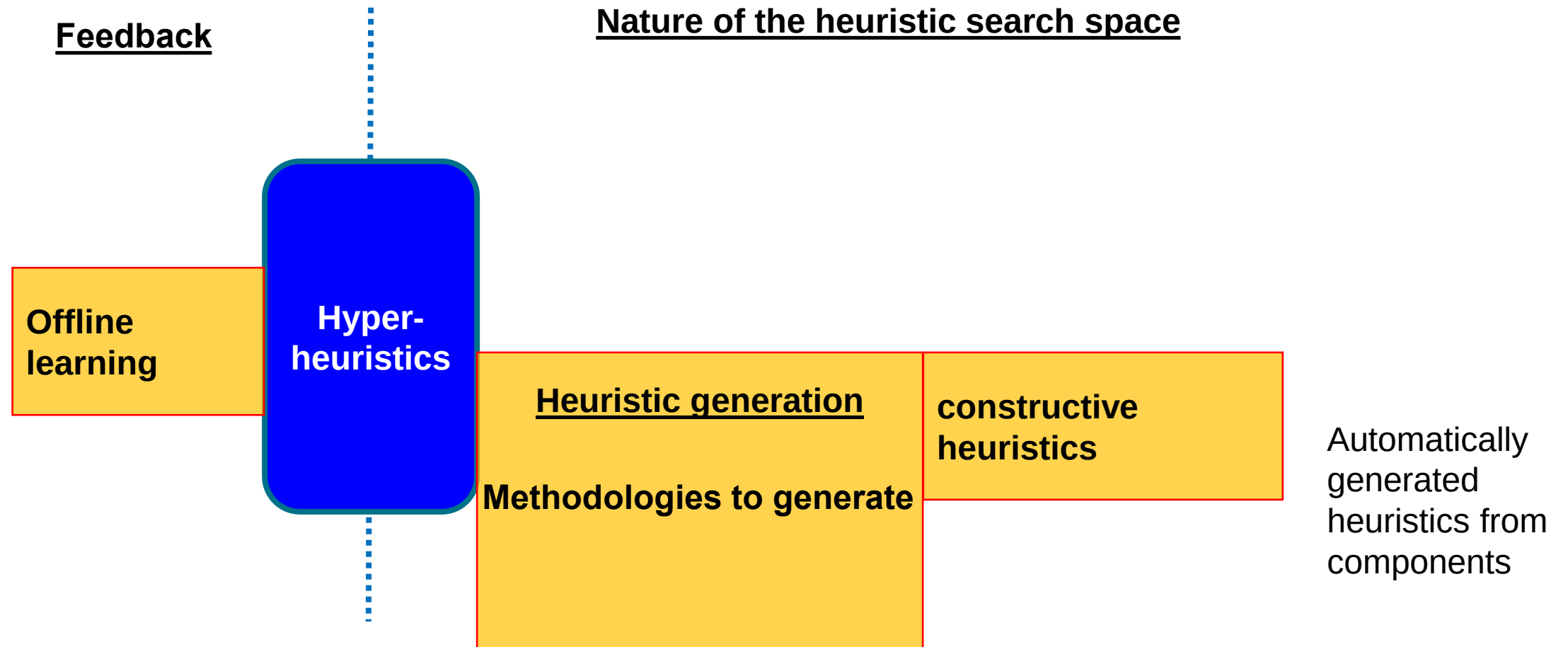
E. K. Burke, M. Hyde, G. Kendall, G. Ochoa, E. Özcan, and J. Woodward (2010). A Classification of Hyper-heuristic Approaches. In Gendreau, Michel and Potvin, Jean-Yves (eds.), Handbook of Metaheuristics, International Series in Operations Research & Management Science, Volume 146, pp. 449-468. Springer. [\[PDF\]](#)



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Our Focus TODAY is...

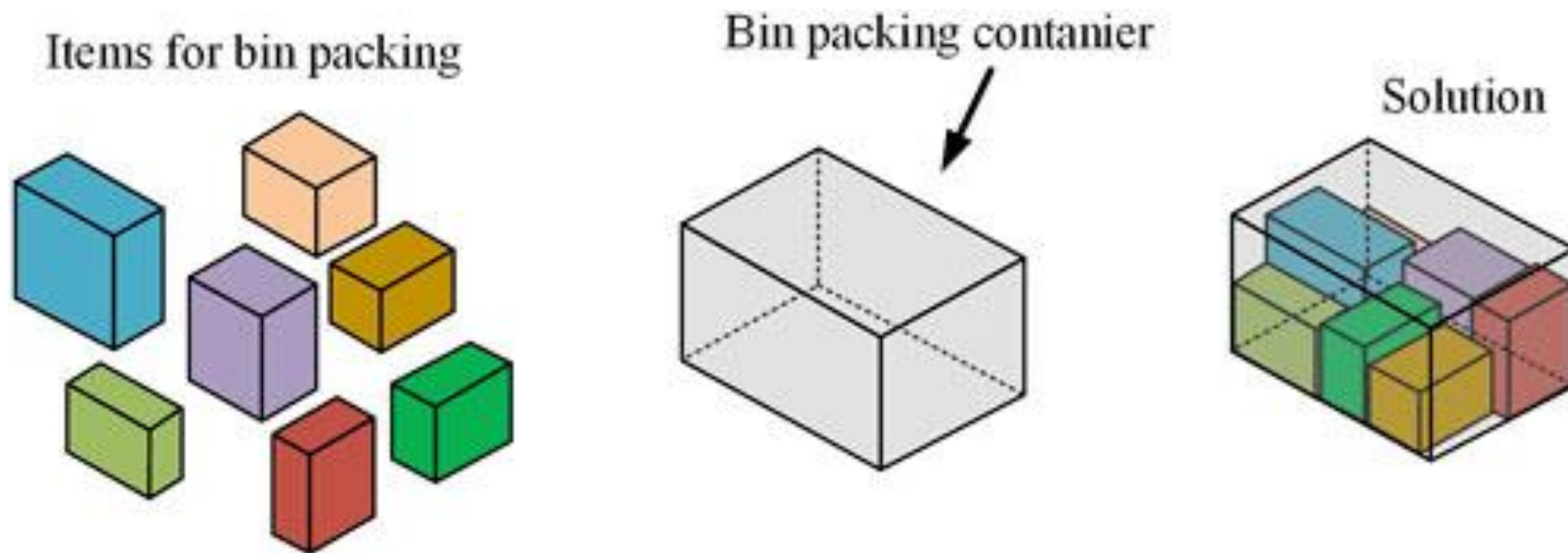


ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



1D Offline Bin Packing

- The **one-dimensional offline bin packing problem (BPP)** is a classic problem that involves packing a set of items into the fewest bins possible while not exceeding the capacity of each bin



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



1D Offline Bin Packing

Pack a **set** of items of sizes s_i for $i = 1, \dots, n$

- Sizes are integer values and $s_i \in [1, C]$
- C is the fixed capacity of each bin

in such a way that

- Never exceed bin capacity
- Minimise number of bins used

Standard NP-hard problem



1D Online Bin Packing

Pack a **stream** of items of sizes s_i for $i = 1, \dots$

- Sizes are integer values and $s_i \in [1, C]$
- C is the fixed capacity of each bin

upon their arrival (one item at a time)

in such a way that

- Never exceed bin capacity
- Minimise number of bins used (Maximise the average bin-fullness) **at the end**



Heuristics for Bin Packing

- **Online Bin Packing:** Heuristics like First-Fit, Next-Fit, and Best-Fit are preferred due to their quick decision-making capabilities.
- **Offline Bin Packing:** Heuristics such as Best-Fit Decreasing and First-Fit Decreasing are more suitable as they can leverage the full set of items to optimize packing.



Heuristics for Bin Packing

- **First-Fit (FF):**
 - Place each item in the first bin that has enough remaining capacity.
 - If no existing bin can accommodate the item, a new bin is opened.
- **Best-Fit (BF):**
 - Place each item in the bin that will leave the least remaining space after the item is added.
 - This heuristic aims to minimize wasted space in bins.



Heuristics for Bin Packing

- **Worst-Fit (WF):**
 - Place each item in the bin that currently has the most remaining space.
 - This approach is intended to keep larger bins available for future items.
- **Next-Fit (NF):**
 - Similar to First-Fit, but once a bin is filled, the algorithm moves on to the next bin without reconsidering previous bins.



Heuristics for Bin Packing

- **First-Fit Decreasing (FFD):**
 - Sort items in decreasing order before applying the First-Fit heuristic.
 - This often leads to better packing performance than the standard First-Fit approach.
- **Best-Fit Decreasing (BFD):**
 - Sort items in decreasing order and then apply the Best-Fit heuristic.
 - Like FFD, this generally yields better results compared to standard Best-Fit.
- **Greedy Algorithms:**
 - Employ greedy strategies that make local optimal choices to build up a global solution without backtracking.

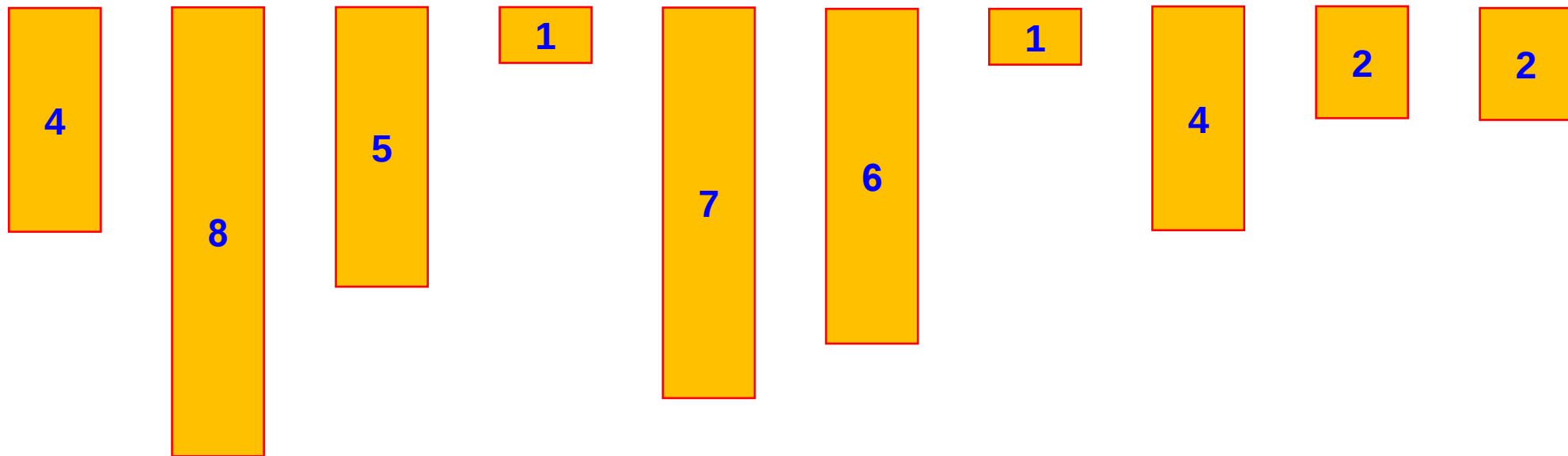


Standard Heuristics: **First-fit** and **Best-fit**

First Fit Heuristic (BP-FF). Place the items in the order in which they arrive. Place the next item into the lowest numbered (left most) bin in which it fits. If it does not fit into any open bin, start a new bin.

Best Fit Heuristic (BP-BF). Place the items in the order in which they arrive. Place the next item into that bin which will leave the least room left over after the item is placed in the bin. If it does not fit in any bin, start a new bin.

$$S = \{4, 8, 5, 1, 7, 6, 1, 4, 2, 2, \dots\} \quad C = 10$$

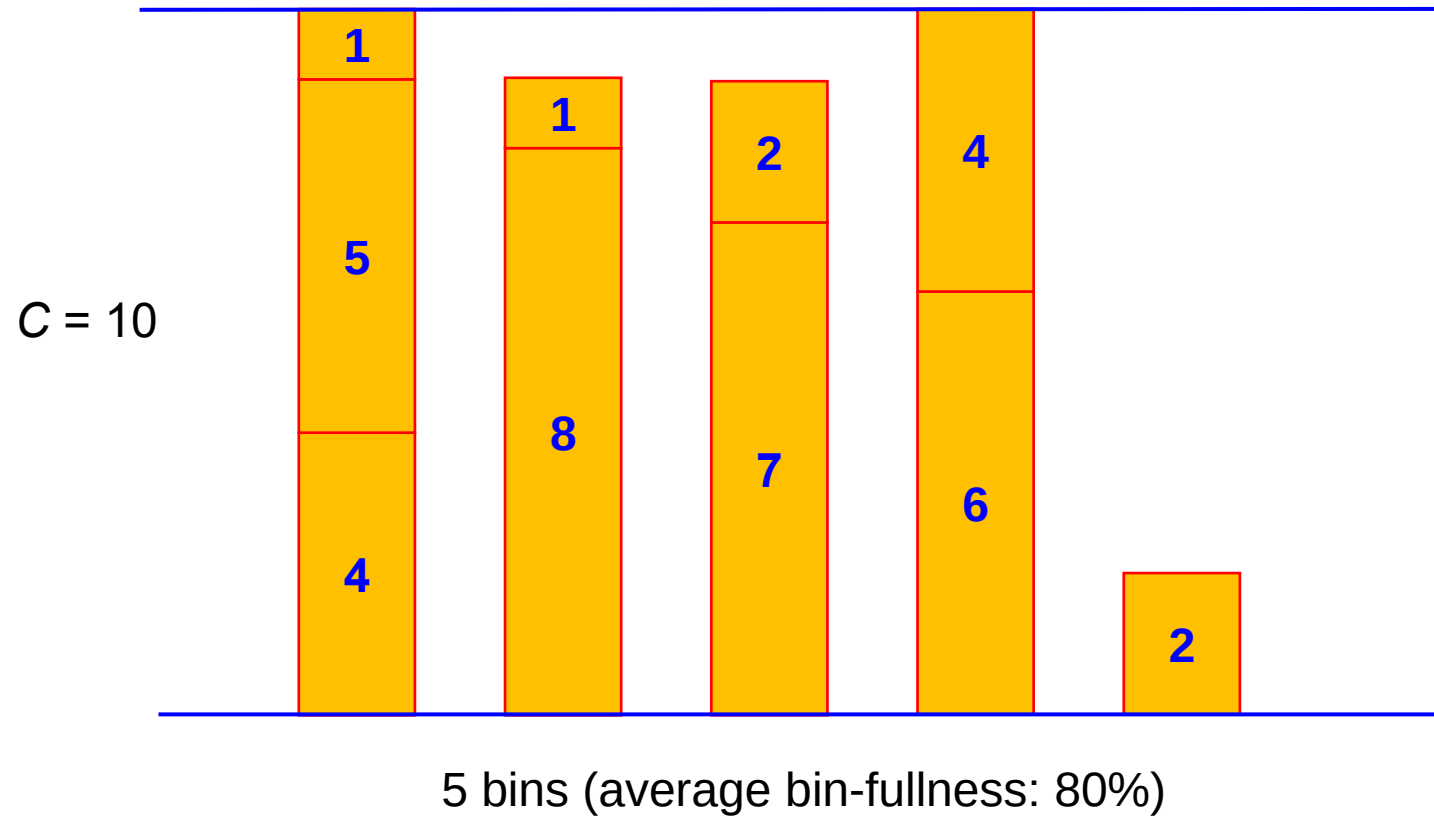


ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Standard Heuristics: **First-fit** and **Best-fit**

$S = \{4, 8, 5, 1, 7, 6, 1, 4, 2, 2, \dots\}$ $C = 10$ **Online Case**

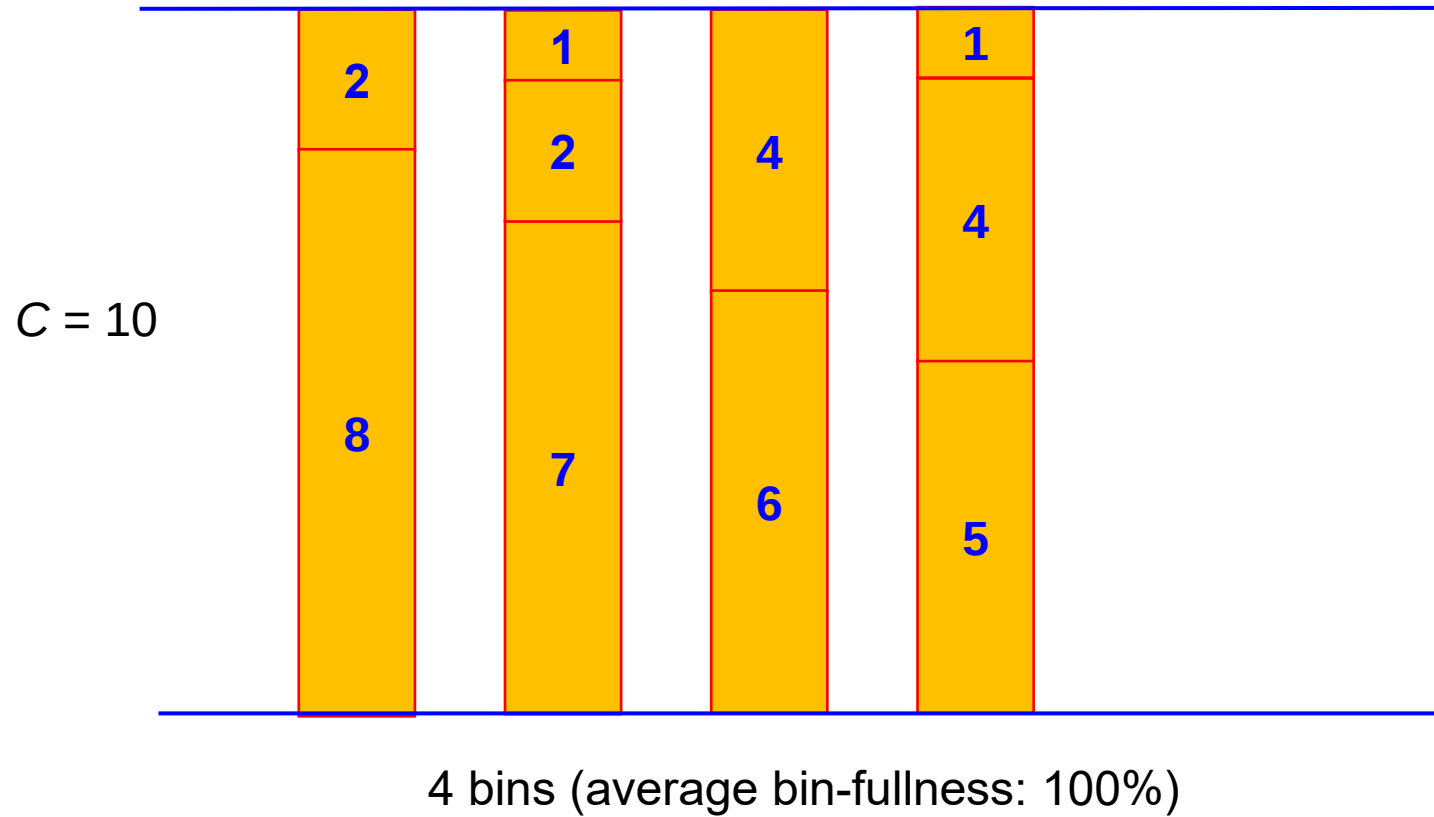


ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Standard Heuristics: **First-fit** and **Best-fit**

$S = \{8, 7, 6, 5, 4, 4, 2, 2, 1, 1\}$ $C = 10$ **Online Case**



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Creating Heuristics via Many Parameters' (CHAMP) for online bin packing

- **Heuristics** are usually designed by an expert through a trial and error process particularly for solving the problems from a specific domain.
- Given the time-consuming nature of the overall process, automated generation of heuristics has been of interest to both academics and practitioners.



Creating Heuristics via Many Parameters' (CHAMP) for online bin packing

- “index policy”: a decision-making framework where each potential choice option is assigned a score or “index value.”
- ‘policy matrix’ (a (large) matrix of ‘heuristic scores’) defines an ‘index policy’ covering the possible observed states.
- A potential decision is given a score independently of other decisions and the decision with the largest score based on a given state is selected.



“Index Policies”

- “**index policy**”: each choice option is given a score, or “index value” independently of the other options
 - The option with the highest index value is taken
 - Also need a rule to break ties
- Although index policies are a special case, in some situations, they can be optimal, or at least very good



Index Policy

1. **Scoring Mechanism:** Each decision option is assigned an independent score or "index value."
2. **Selection Criteria:** The option with the highest index value is selected for execution.
3. **Tie-Breaking:** A predefined rule is necessary to resolve situations where multiple options have the same score.
4. **Performance:** Index policies can be optimal or yield good results in specific scenarios, making them effective for decision-making tasks.



Index Policies for 1D Online Bin Packing

- Given
 - r : remaining capacity of bin (residual space)
 - s : item size
- score of bin is $f(r,s)$ for some function f
- Given a new item of size then place into bin with largest value of $f(r,s)$
- We will break any ties using FF:
 - **place item in earliest bin with the best available score**



Index Policies for 1D Online Bin Packing

Algorithm 1: Applying a policy matrix on a bin packing instance.

```
1 In :  $W$  :score matrix;
2 for each arriving item size  $s$  do
3    $maximumScore = 0$ ;
4   for each open bin  $i$  in the list with remaining size  $k$  do
5     if  $k > s$  then
6       if  $W_{k,s} > maximumScore$  then
7          $maximumScore = W_{k,s}$ ;
8          $maximumIndex = i$ ;
9       end
10    end
11  end
12  assign the item to the bin  $maximumIndex$ ;
13  if  $maximumIndex$  is the empty bin then
14    open a new empty bin and add to the list;
15  end
16  update the remaining capacity of  $maximumIndex$  by
    subtracting  $s$  from it;
17  if remaining capacity of  $maximumIndex$  is none then
18    close the bin  $maximumIndex$ ;
19  end
20 end
```

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul



Policy Matrix

- A policy matrix is a structured representation of decision options where each cell corresponds to a score for a specific choice based on the current state.
- The matrix serves as a heuristic with multiple parameters, allowing for fine-tuned decision-making.
- An online bin packing simulator acts as the evaluation function to assess the effectiveness of the policy matrix.



Policy matrix representation

- The online bin packing instances produced by a parametrised stochastic generator are represented by the formalism: $UBP(C, s_{\min}, s_{\max}, N)$ where
 - C = bin capacity
 - $s_{\min}$ and $s_{\max}$ = minimum and maximum item sizes
 - N is the total number of items.



Policy matrix representation

r\s	5	6	7	8	9	10
5:	11	.	.	.	.	.
6:	10	8	.	.	.	.
7:	2	3	10	.	.	.
8:	8	5	5	7	.	.
9:	8	11	1	3	3	.
10:	4	1	4	7	5	2
11:	4	9	1	1	8	3
12:	3	4	6	3	1	1
13:	7	11	1	3	4	7
14:	12	10	8	2	7	1
15:	1	2	4	8	5	2
16:	.	.	.	.	.	.
17:	.	.	.	.	.	.
18:	.	.	.	.	.	.
19:	.	.	.	.	.	.
20:	4	9	4	2	2	5

- W represents a policy matrix for a given UBP.
- $W_{r,s}$ is an integer score at the r^{th} row and s^{th} column of the policy matrix W , where $s_{min} \leq s \leq s_{max}$.
- $W_{r,s} \in [w_{min}, w_{max}]$, that is, $1 \leq w_{min} \leq W_{r,s} \leq w_{max} \leq C$.

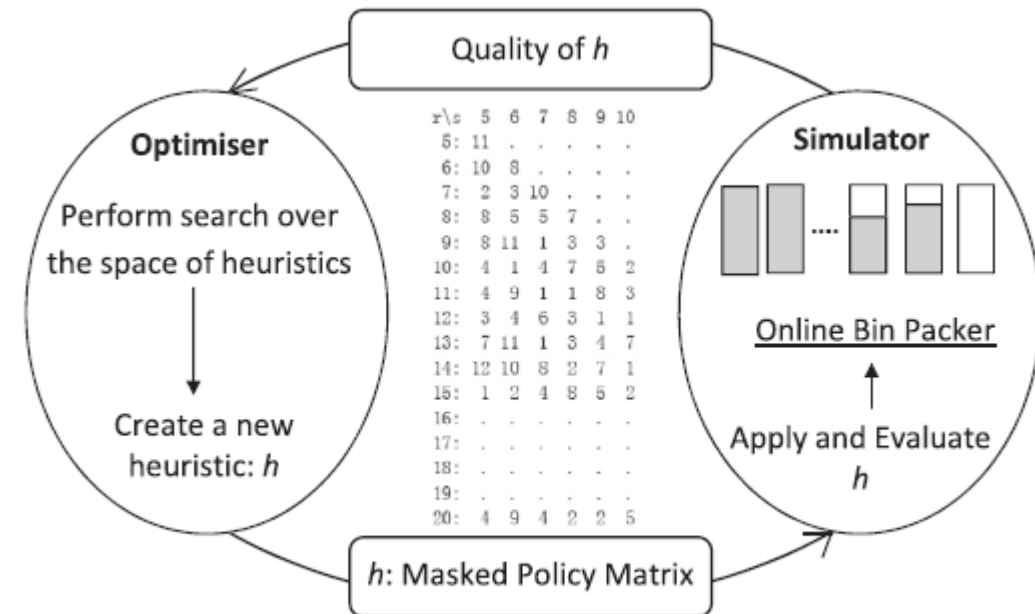


Fig. 1. An example of a policy matrix for UBP(20, 5, 10, 10^5).

Fig. 2. CHAMP framework for the online bin packing problem.



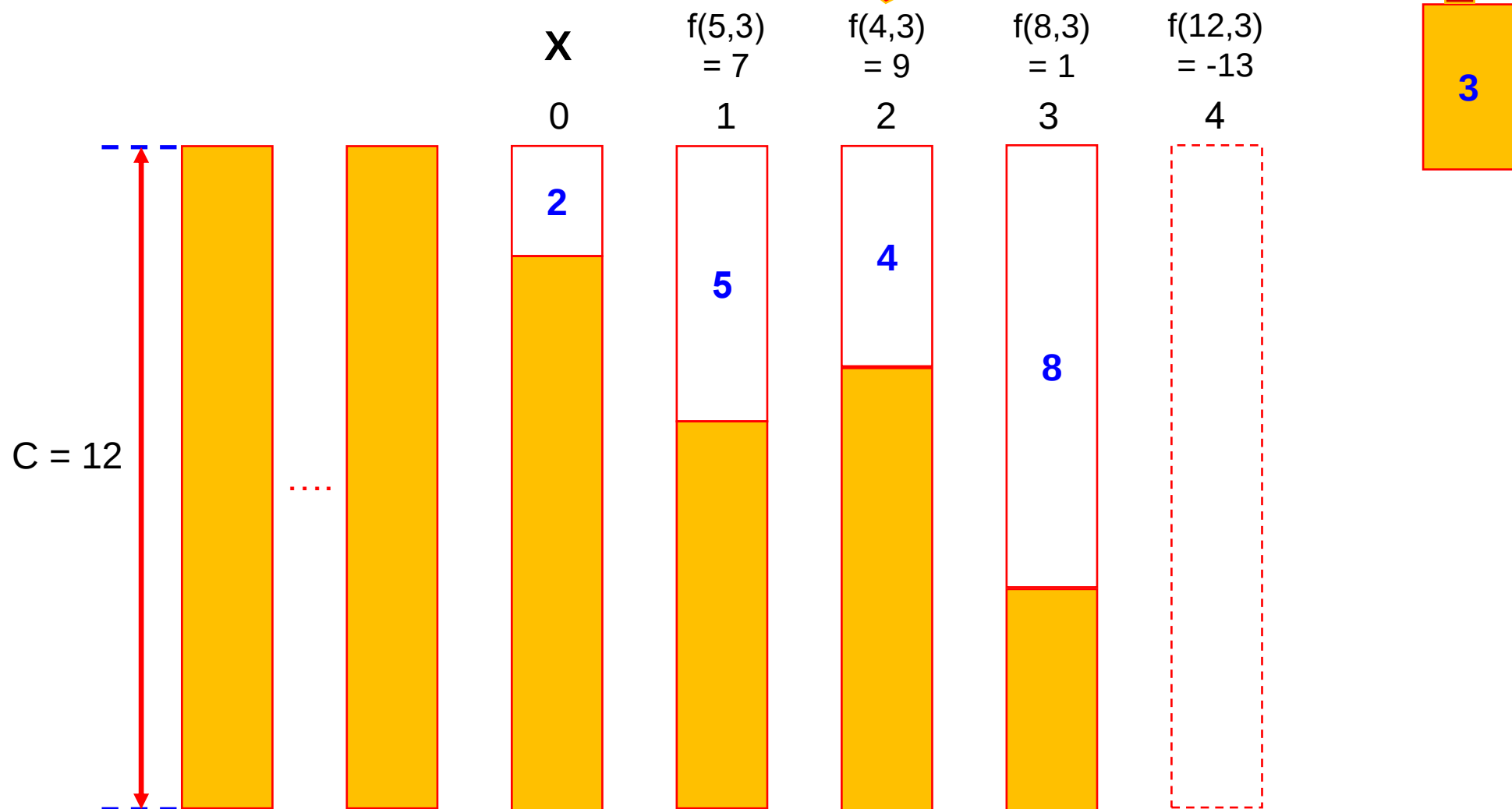
Policy matrix representation

- Each row of the W matrix corresponds to a specific state in the online bin packing problem, while each column represents a possible decision option (i.e., which bin to use for packing an item).
- For each state, a score is assigned to each decision option based on the remaining capacity of the bins and the characteristics of the incoming item. This score reflects how well that option would fit the current scenario.
- The scoring mechanism considers factors such as:
 - The remaining space in each open bin.
 - The size of the item being packed.
 - The overall goal of maximizing bin fullness.

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



1D Bin Packing – Intermediate Step



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



1D One Bin Packing

- A new empty bin is always available (**open**)
- A bin is **closed** if it takes no more items
 - e.g., if **residual space is smaller than size of any item**
- We need a *good* “policy”, i.e., a method to assign a new item upon its arrival to one of the *open* bins



Potential General Method for 1D Online Bin Packing

- On arrival of new item of size s_i
 - Inspect the current set of open bins
 - Simultaneously use the entire set of residual spaces in the open bins to pick where to place the new item
- This is difficult and expensive (in general)



Generating Heuristics

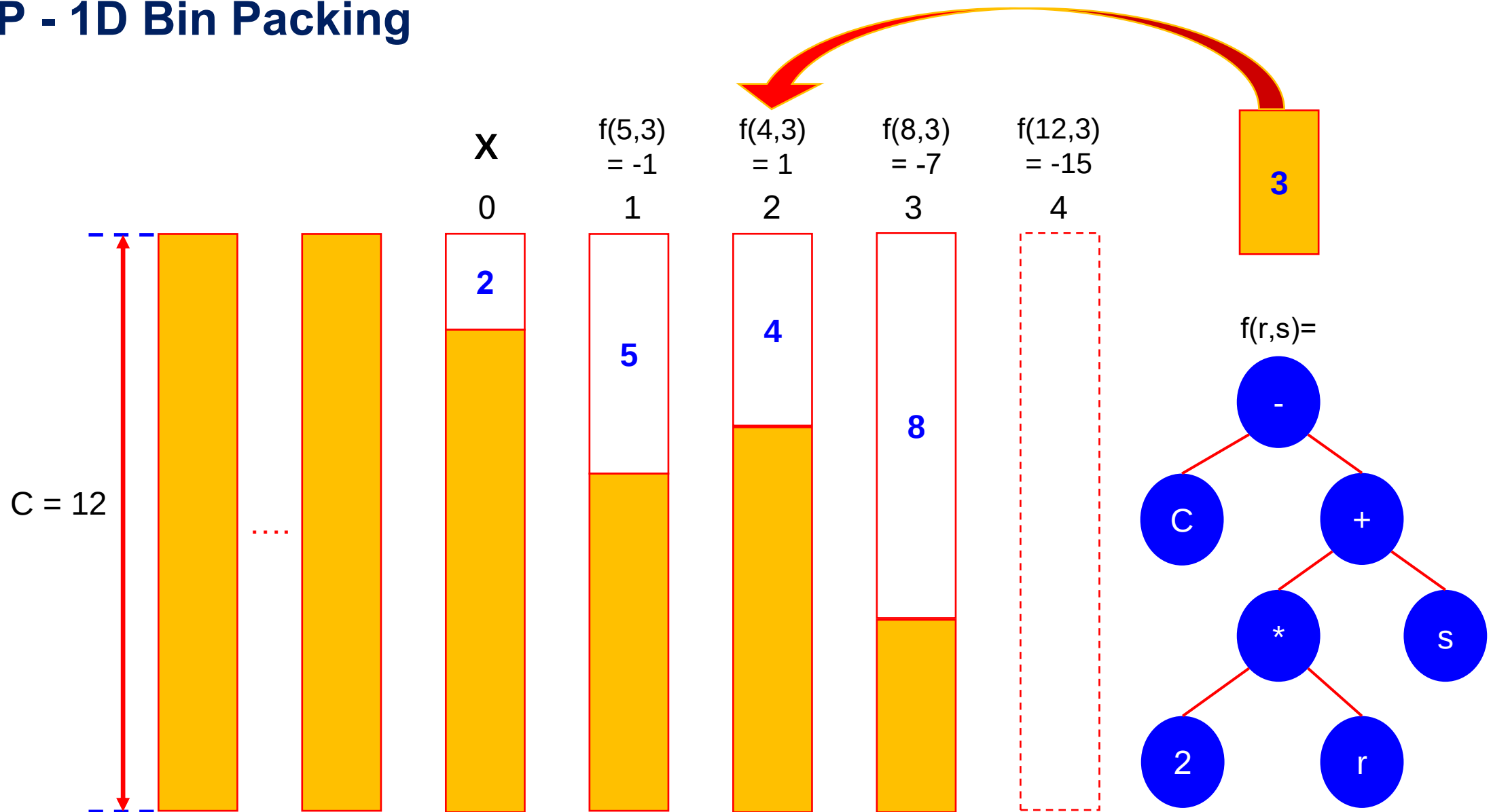
- Within search methods, often have score functions, “index functions” to help make some choice
 - difficult to invent successful ones; want to automate this
- GP approach: evolve arithmetic score functions

E. K. Burke, M. R. Hyde, G. Kendall, J. R. Woodward: Automatic heuristic generation with genetic programming: evolving a jack-of-all- trades or a master of one. GECCO 2007: 1559-1565 [[PDF](#)]

- Use *Genetic Programming* to learn $f(r,s)$
- $f(r,s)$ is represented as arithmetic function tree
- Automatically creates functions that at least match FF, BF



GP - 1D Bin Packing



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Generating Heuristics

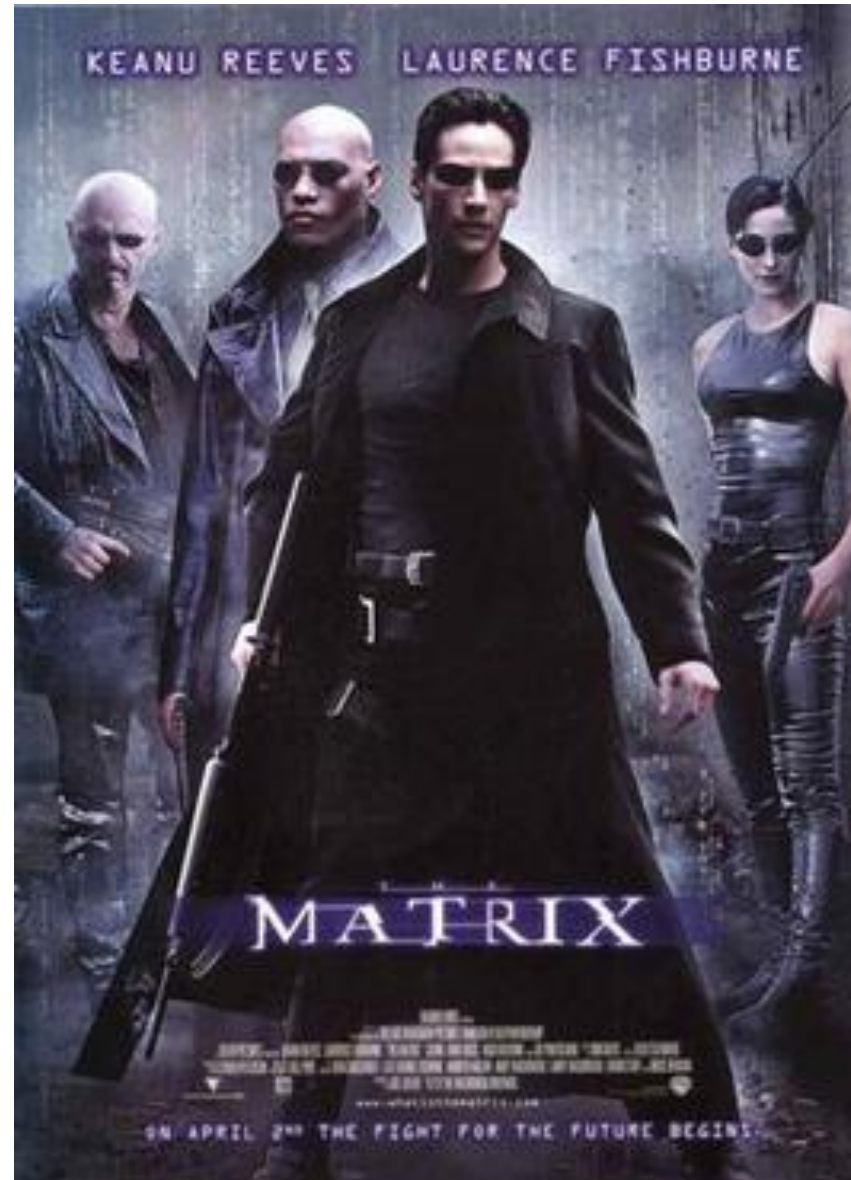
CHALLENGE:

- Space of functions, as used in GP,
 - is hard to understand
 - potentially biased because of the choice of representation
 - some perfectly good functions might have “bloated” representations

Can one do even better?



Is there another way to find policies?



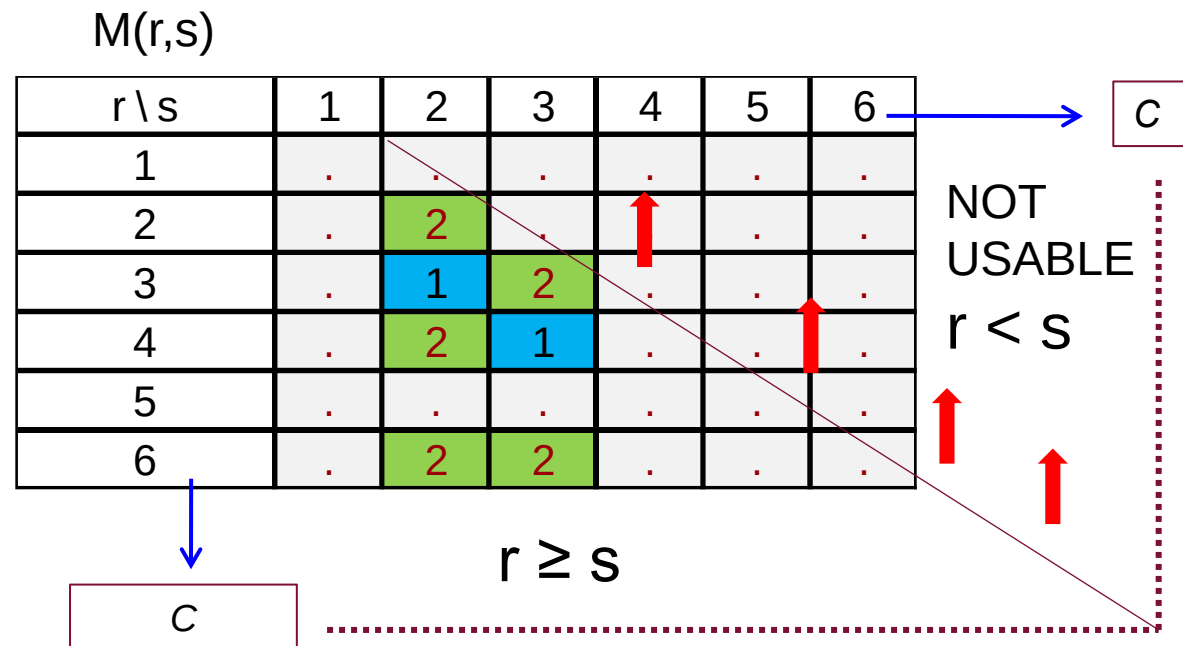
ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Source: http://en.wikipedia.org/wiki/The_Matrix



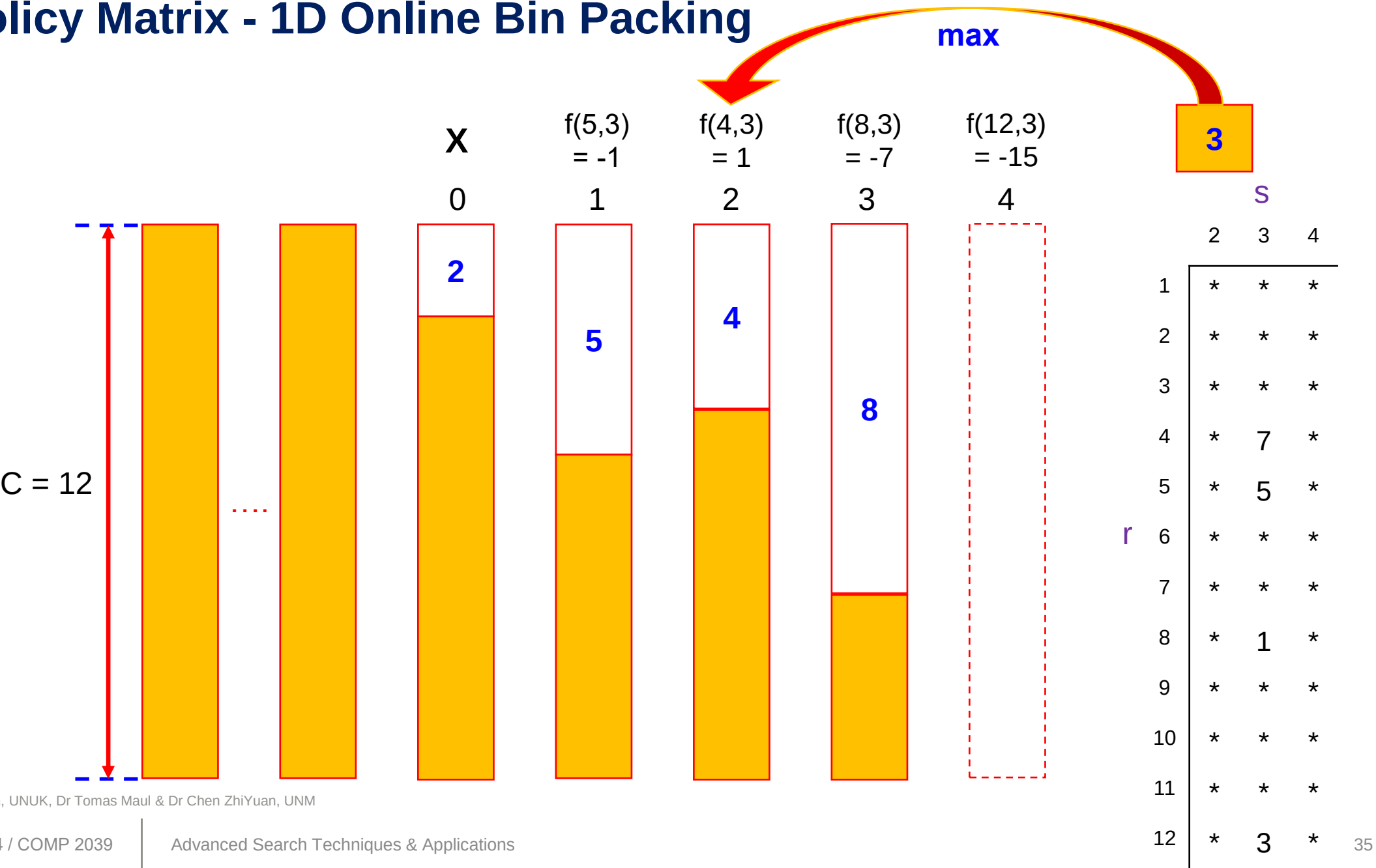
Matrix View of Policies/Heuristics

- Since all item sizes (s) and residual capacities (r) are integer, then $f(r,s)$ is simply a large ($C \times C$) matrix $M(r,s)$ of **parameter values**





Policy Matrix - 1D Online Bin Packing



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Uniform (**random**) Instances

We empirically studied matrix policies on Uniform Bin Packing problems

$\text{UBP}(C, s_{\min}, s_{\max}, N)$ (problem generator)

- Bin capacity C
- N items are generated with integer sizes independently taken uniformly at random from the range $[s_{\min}, s_{\max}]$
 - N is usually taken to be large: 100k



UBP (6, 2, 3)

- (Bin capacity 6, items are size 2 or 3 only.)
- The only perfect packings are
 - $2+2+2$
 - $3+3$
- Hence the ‘obvious’ policy is ...
- ... “*never mix even and odd sizes*”



UBP (6, 2, 3)

- ... “never mix even and odd sizes”
- Index policy as a matrix:
 - rows: residual capacity of the bin
 - columns: item size

resid \ item	1	2	3	4	5	6
1	.	.	.	.	.	.
2	.	2	.	.	.	.
3	.	1	2	.	.	.
4	.	2	1	.	.	.
5	.	.	.	.	.	.
6	.	2	2	.	.	.

- Ties are broken using First-Fit (FF)
- Grey entries “.” are never usable

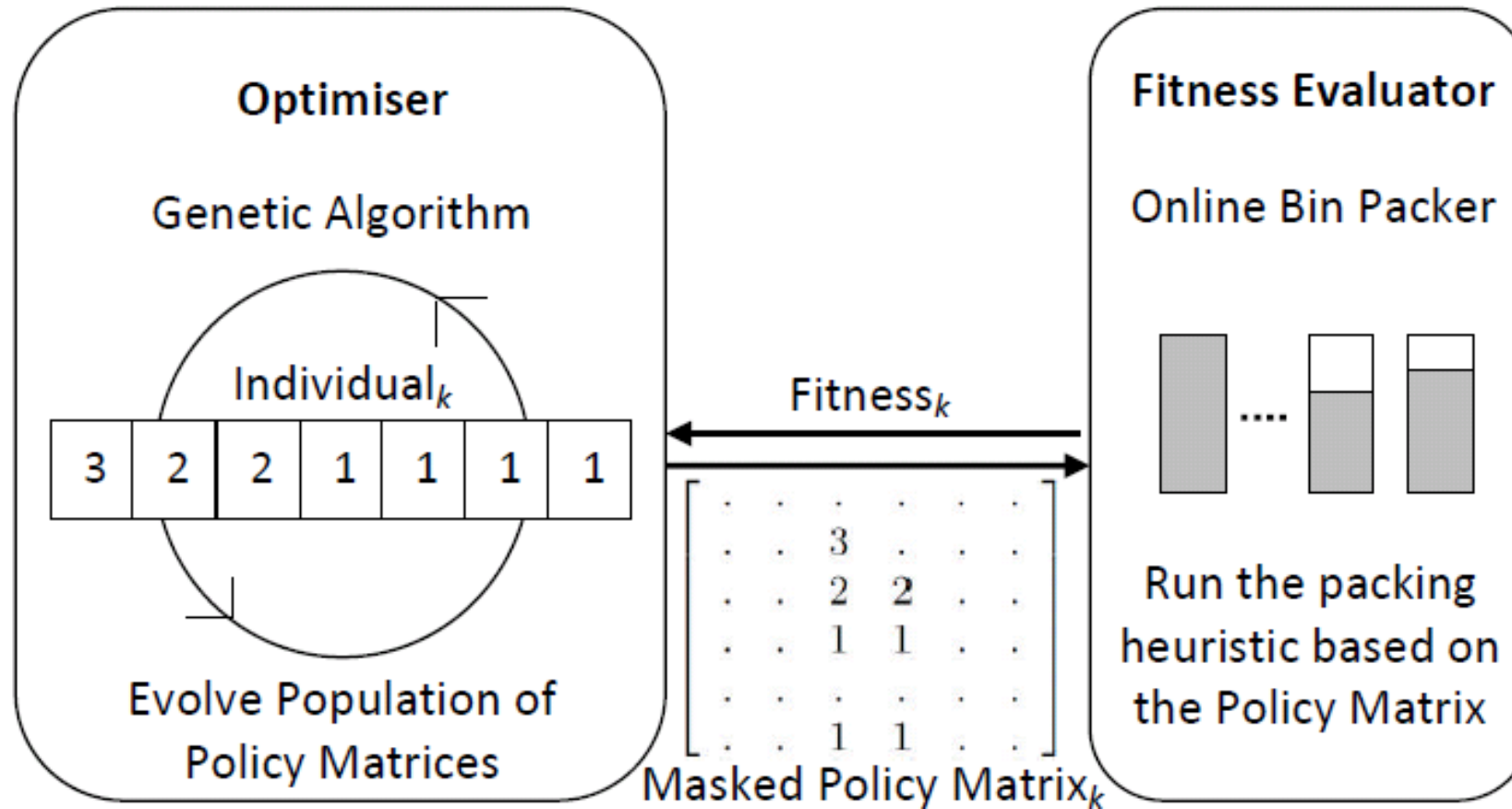


Creating Heuristics via Many Parameters - CHAMP

- Basic idea:
 - Take values in matrix $M(r,s)$ to be integers
 - Do (meta-)heuristic search to find good choices for $M(r,s)$: Evaluation is by simulation
- Our Original Expectation:
 - the matrix will tweak the functions from GP and might slightly improve performance
- Potential expected disadvantages:
 - matrices can be much more verbose than functions
 - they fail to take into account of the good structure captured by functions



CHAMP-GA Architecture

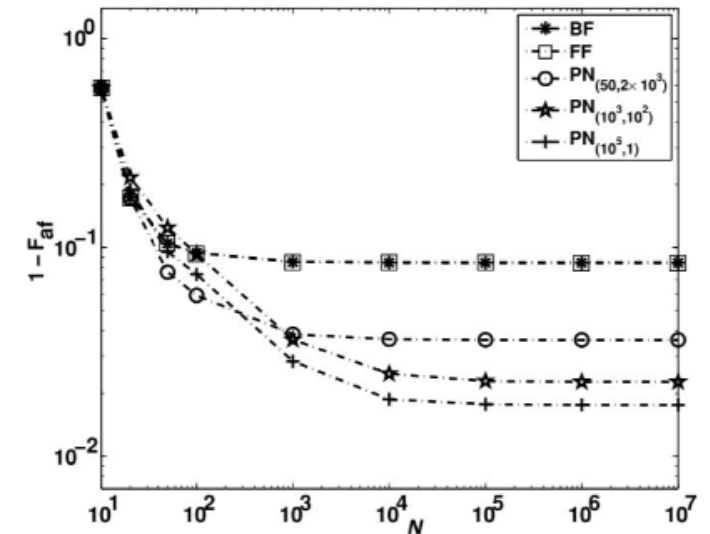


ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



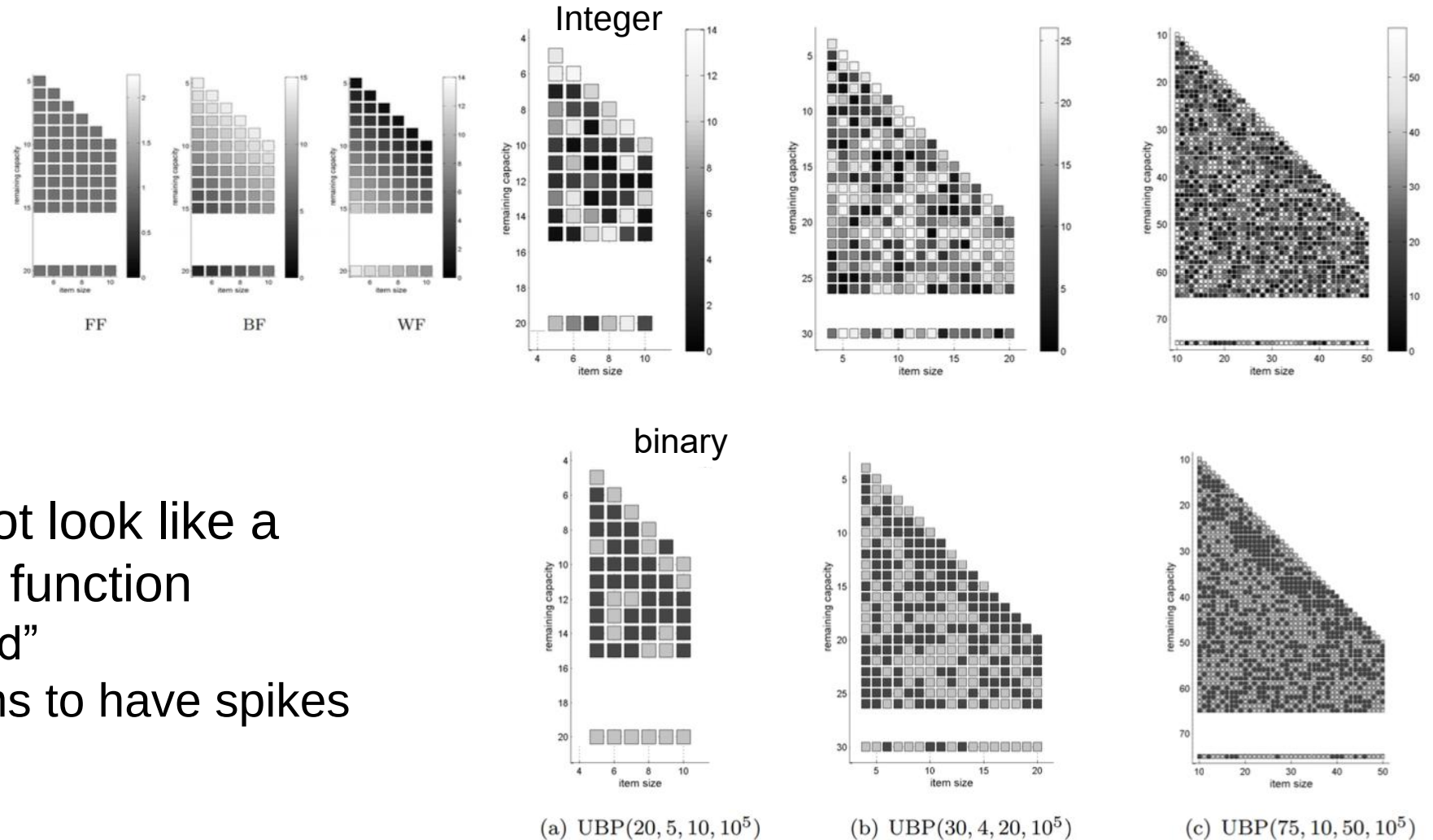
Implementation Details

- Apply a standard GA for training
 - Trans-generational (with weak elitism, population size: $C/2$), tournament selection (tour size: 2), Uniform Crossover, standard mutation (with probability $1/L$), termination after 200 iterations, number of trials: 1.
- Only the active members of the matrix are stored as integer/binary values ($GA_{Original}/GA_{Binary}$) in the chromosome (GA_{FFinit} : $GA_{Original}$ where initial population contains -seeded with- an individual representing FF)
- Evaluation:
 - write matrix to a file
 - use matrix as input for a program that packs many items





Examples of good evolved matrices



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Results – Best

Method	UBP(6, 2, 3, 10 ⁵)	UBP(15, 5, 10, 10 ⁵)	UBP(20, 5, 10, 10 ⁵)	UBP(30, 4, 20, 10 ⁵)	UBP(30, 4, 25, 10 ⁵)	UBP(40, 10, 20, 10 ⁵)	UBP(60, 15, 25, 10 ⁵)	UBP(75, 10, 50, 10 ⁵)	UBP(80, 10, 50, 10 ⁵)	UBP(150, 20, 100, 10 ⁵)
BF	92.30	99.62	91.55	96.84	98.38	90.23	92.55	96.08	96.39	95.82
FF	92.30	99.55	91.54	96.68	97.93	90.22	92.55	95.91	96.29	95.64
WF	91.70	86.58	90.54	88.61	84.10	88.66	90.80	87.94	89.25	87.73
Harmonic	-	74.24	90.04	73.82	74.21	89.10	85.18	71.59	72.96	71.97
<i>GA_{Original}</i>	99.99	99.63	98.18	99.41	98.39	96.99	99.68	98.22	98.54	97.88
<i>GA_{FFinit}</i>	99.99	99.61	98.15	99.10	99.25	96.05	98.28	98.43	97.87	96.92
<i>GA_{Binary}</i>	99.99	99.61	98.42	99.58	99.55	96.75	96.96	98.45	98.46	97.63

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Conclusions

- Can use standard metaheuristics to create policies expressed in matrix representation
 - Policies exist that out-perform standard heuristics
 - Finding the policies is easier than expected
 - There are many different policies with similar performance
 - The policies are “weirder” than expected, even after smoothing.
 - The good policies could have “random” structures
 - Not necessarily easy to capture with an algebraic function of GP
 - The results can be “analysed” (inspected) to produce simple policies that out-perform standard ones
 - and that then scale to larger problems



References

- E. K. Burke, M. Hyde, G. Kendall, G. Ochoa, E. Özcan, and J. Woodward (2009). Exploring hyper-heuristic methodologies with genetic programming. In C. Mumford and L. Jain (eds.), Computational Intelligence, Intelligent Systems Reference Library, pp. 177-201. Springer [[PDF](#)]
- E. Özcan, and A. J. Parkes, Policy Matrix Evolution for Generation of Heuristics, Proceedings of the 13th Annual Conference on Genetic and Evolutionary Computation (GECCO '11), Natalio Krasnogor (Ed.). ACM, New York, NY, USA, pp. 2011-2018 (won the best paper award in the Self-* track), 2011 [[original-PDF](#)]. [[PDF](#)]
- A. J. Parkes, E. Özcan, M. Hyde, Matrix Analysis of Genetic Programming Mutation, EuroGP 2012, Lecture Notes in Computer Science 7244, pp. 158-169, 2012. [[PDF](#)]
- S. Asta, E. Özcan and A. J. Parkes, Dimension reduction in the search for online bin packing policies, Proceedings of the 2013 Genetic and Evolutionary Computation Conference Companion, pp. 65-66, 2013. [[PDF](#)]
- S. Asta, E. Özcan, and A.J. Parkes, CHAMP: Creating Heuristics via Many Parameters for Online Bin Packing, Expert Systems With Applications, vol. 63, pp. 208-221, doi:10.1016/j.eswa.2016.07.005, 2016 [[original PDF](#)]. [[PDF](#)]

ACK: Prof. E. Özcan, UNIK Bonn, Germany, and Prof. A. J. Parkes, UNIK



Break





University of
Nottingham

UK | CHINA | MALAYSIA

Data Science (Machine Learning)

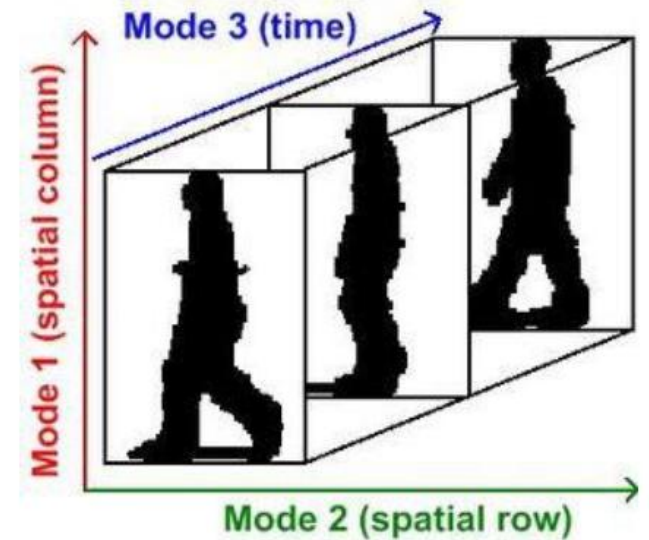
Improved Hyper-Heuristic
Optimisation

S. Asta and E. Özcan, A Tensor-based Selection Hyper-heuristic for Cross-domain Heuristic Search, Information Sciences, vol. 299, pp. 412-432, 2015



Single Objective Hyper²-heuristic: A Data Science Improved Hyper-heuristic

- Many real-world data are *multidimensional*
 - *Very high-dimensional* (big) with a large amount of redundancy
- **Multi-dimensional arrays** representing such data describe a *tensor*
- Many applications in signal processing, psychometrics, social network analysis, genomics and more



S. Asta and E. Özcan, A Tensor-based Selection Hyper-heuristic for Cross-domain Heuristic Search, Information Sciences, vol. 299, pp. 412-432, 2015

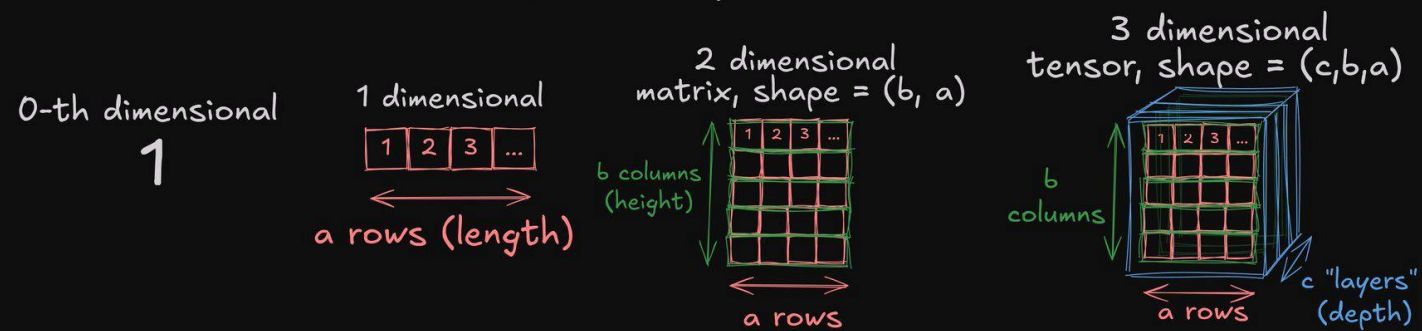


Tensor Factorization

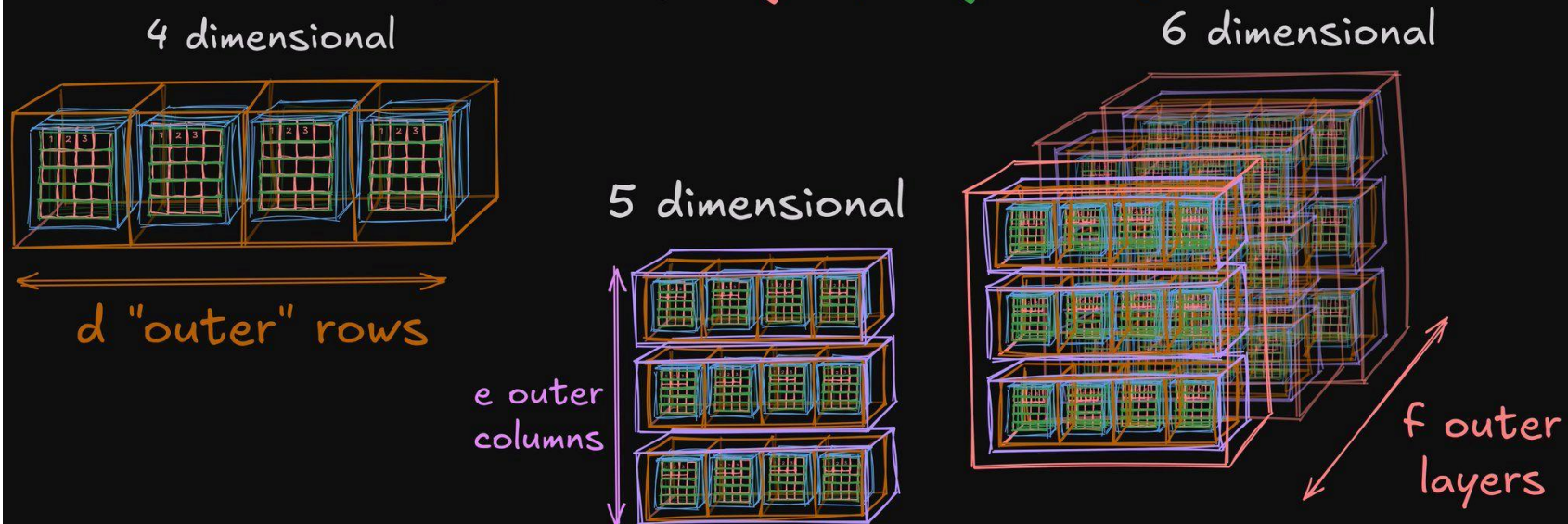
- A **tensor** is a generalization of scalars (0D), vectors (1D), and matrices (2D) to higher dimensions. For example, a 3D tensor can be thought of as a cube of numbers.
- **Tensor factorization** is a technique that breaks down a multidimensional array, or tensor, into factor matrices (in lower-dimensional tensors). The goal is to express the tensor as a combination of lower-dimensional tensors. This helps in extracting latent factors that describe the data.



visualizing hyperdimensional boxes, a quickguide



where do we go from here?
keep stacking **length**, **height**, **depth**!



and so on!



Tensor

A tensor is an N-dimensional array of data



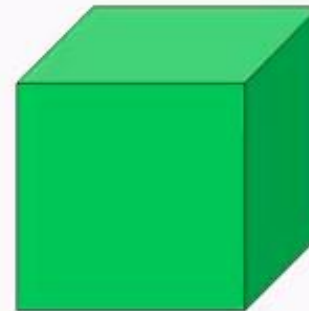
Rank 0
Tensor
scalar



Rank 1
Tensor
vector



Rank 2
Tensor
matrix



Rank 3
Tensor



Rank 4
Tensor

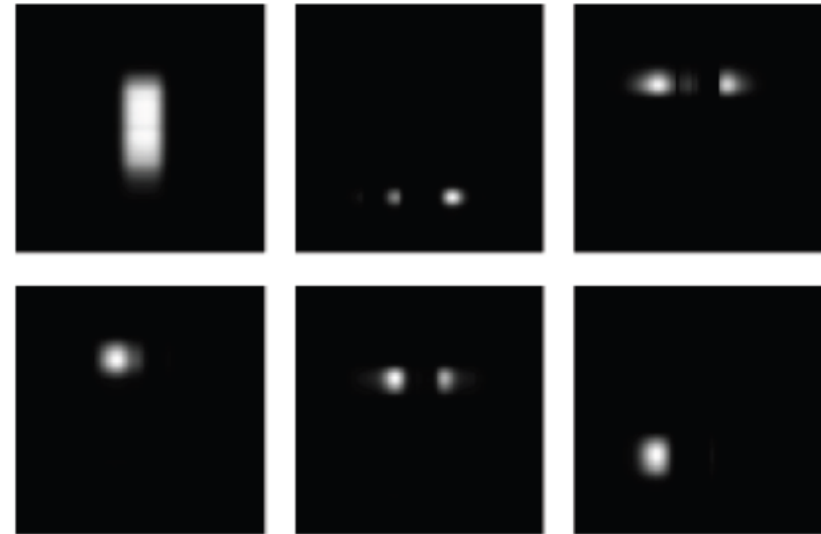


Tensor Factorisation

There are different decomposition methods, we use *Canonical Polyadic (CP) factorisation*

$$\hat{T} = \sum_{k=1}^K \lambda_k a_k \circ b_k \circ c_k$$

- This gives a projection of 3D data onto 1D vectors
- Helps to discover latent structures in data, quantifying the relationship between pairs of different components



SOURCE: B. Krausz, C. Bauckhage, Action recognition in videos using nonnegative tensor factorization., in: ICPR, IEEE, 2010, pp. 1763–1766.



Types of Tensor Factorization

- **CANDECOMP/PARAFAC (CP):** Decomposes a tensor into a sum of rank-one tensors. Each rank-one tensor is the outer product of vectors.
- **Tucker Decomposition:** Generalizes CP by decomposing the tensor into a core tensor multiplied by factor matrices along each mode. This allows for more flexibility in capturing interactions among different dimensions.
- **Tensor Train (TT) Decomposition:** Represents a tensor as a sequence of matrices. It is especially efficient for high-dimensional tensors.



Canonical Polyadic (CP) factorization

- **Canonical Polyadic (CP) factorization**, also known as CANDECOMP/PARAFAC decomposition, is a mathematical technique used to decompose a multi-dimensional tensor into a sum of rank-1 tensors.
- It is an extension of matrix factorization techniques like Singular Value Decomposition (SVD) to higher-dimensional data.



Canonical Polyadic (CP) factorization

Rank-1 Tensor: A tensor that can be written as the outer product of vectors from each of its modes. For example, in a 3D tensor, a rank-1 tensor is the outer product of three vectors:

$$\mathbf{X} \approx \sum_{r=1}^R \mathbf{a}_r \circ \mathbf{b}_r \circ \mathbf{c}_r$$

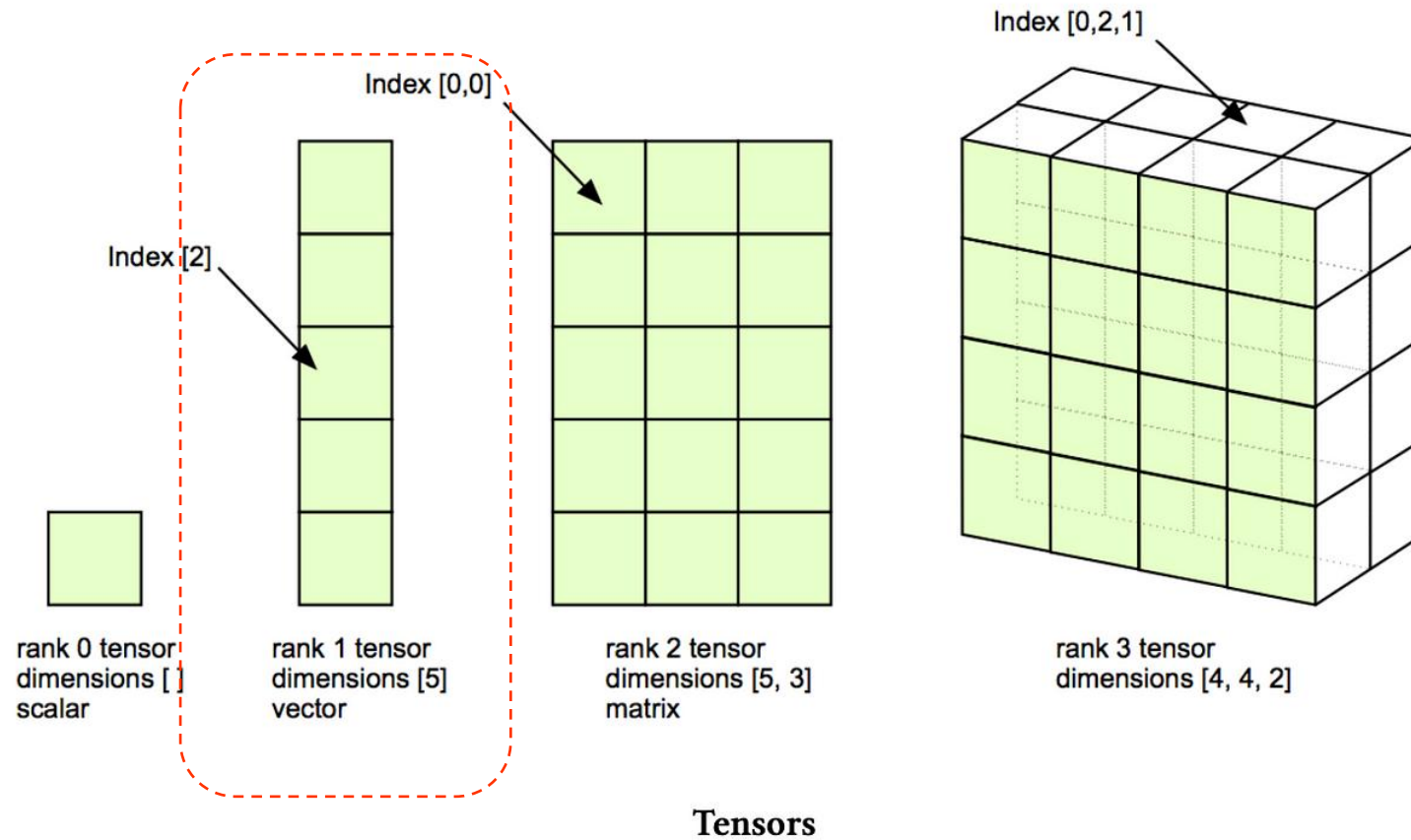
Here:

- R is the rank of the tensor.
- $\circ$ represents the outer product.
- $\mathbf{a}_r, \mathbf{b}_r, \mathbf{c}_r$ are vectors corresponding to the modes of the tensor.

CP Factorization: The process of approximating a tensor $\mathbf{X}$ as a sum of R rank-1 tensors. Each component of the decomposition captures patterns or latent factors in the data along each mode.



Rank-1 Tensors

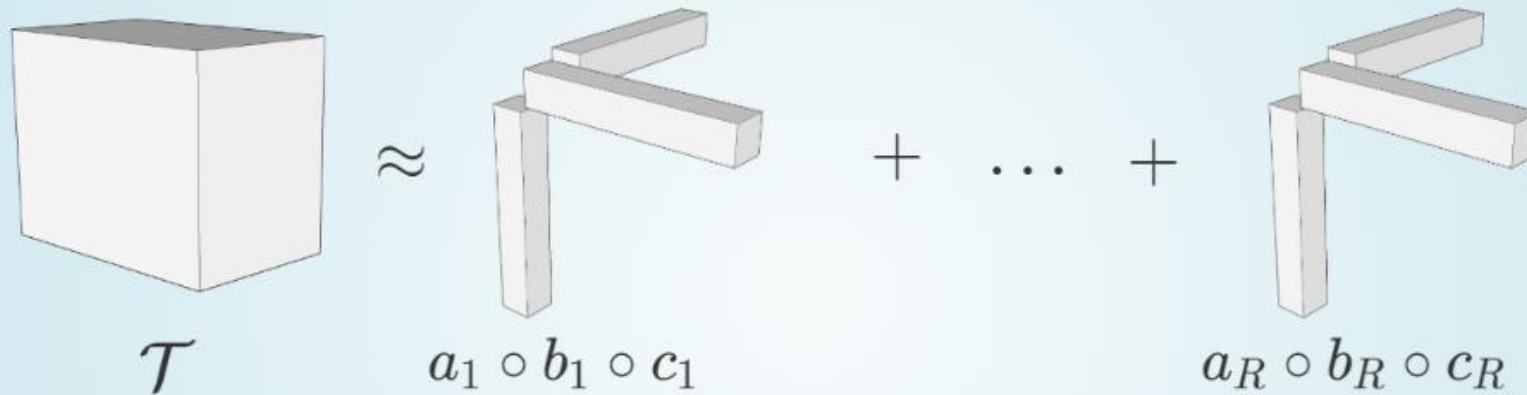


ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Canonical Polyadic (CP) factorization

Canonical Polyadic Decomposition (CPD) is a decomposition of a tensor $\mathcal{T}$ into sum of rank-1 tensors


$$\mathcal{T} \approx a_1 \circ b_1 \circ c_1 + \dots + a_R \circ b_R \circ c_R$$

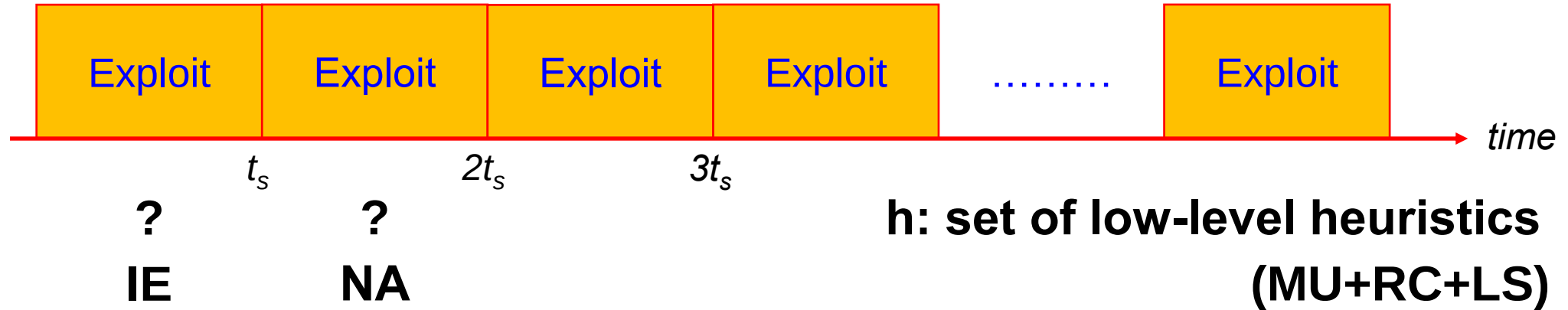
The rank (R) of a CPD is the number of rank-1 tensors used to approximate $a_i, b_i, c_i \geq 0$

$$\min_{\mathbf{a}_i \geq 0, \mathbf{b}_i \geq 0, \mathbf{c}_i \geq 0} \sum_{i=1, \dots, R} \|\mathcal{T} - \mathbf{a}_i \circ \mathbf{b}_i \circ \mathbf{c}_i\|_F^2$$



Proposed Approach - Ideas

The balance between exploration and exploitation is crucial (e.g. ILS)



$$h_{IE} \cup h_{NA} = h \quad (h_{IE} \cap h_{NA} = \emptyset)$$

- Mix move acceptance methods
- Use machine learning to partition the low level heuristics associated with each method



Proposed Approach - Ideas

- Move Acceptance Methods:
 - **IE (Improvement and Equal):** Accepts solutions that improve or maintain the current objective value.
 - **NA (Naive Acceptance):** Accepts improving and equal solutions, but allows worsening solutions with a probability.
- The approach involves using machine learning methods to partition low-level heuristics, optimizing their association with the appropriate move acceptance methods.



Proposed Approach - Ideas

- MU+RC+LS refers to different types of low-level heuristics used in the hyper-heuristic framework
- **MU (Mutation):**
 - This type of heuristic involves altering a solution slightly to explore nearby solutions in the solution space. It introduces variability and can help in escaping local optima.
- **RC (Ruin and Recreate):**
 - This heuristic involves "ruining" a current solution by removing parts of it and then "recreating" it. This approach can lead to discovering new regions in the solution space that may yield better solutions.
- **LS (Local Search):**
 - Local search heuristics focus on iteratively improving a given solution by making small changes and accepting those that lead to better solutions. It's a common method for refining solutions.

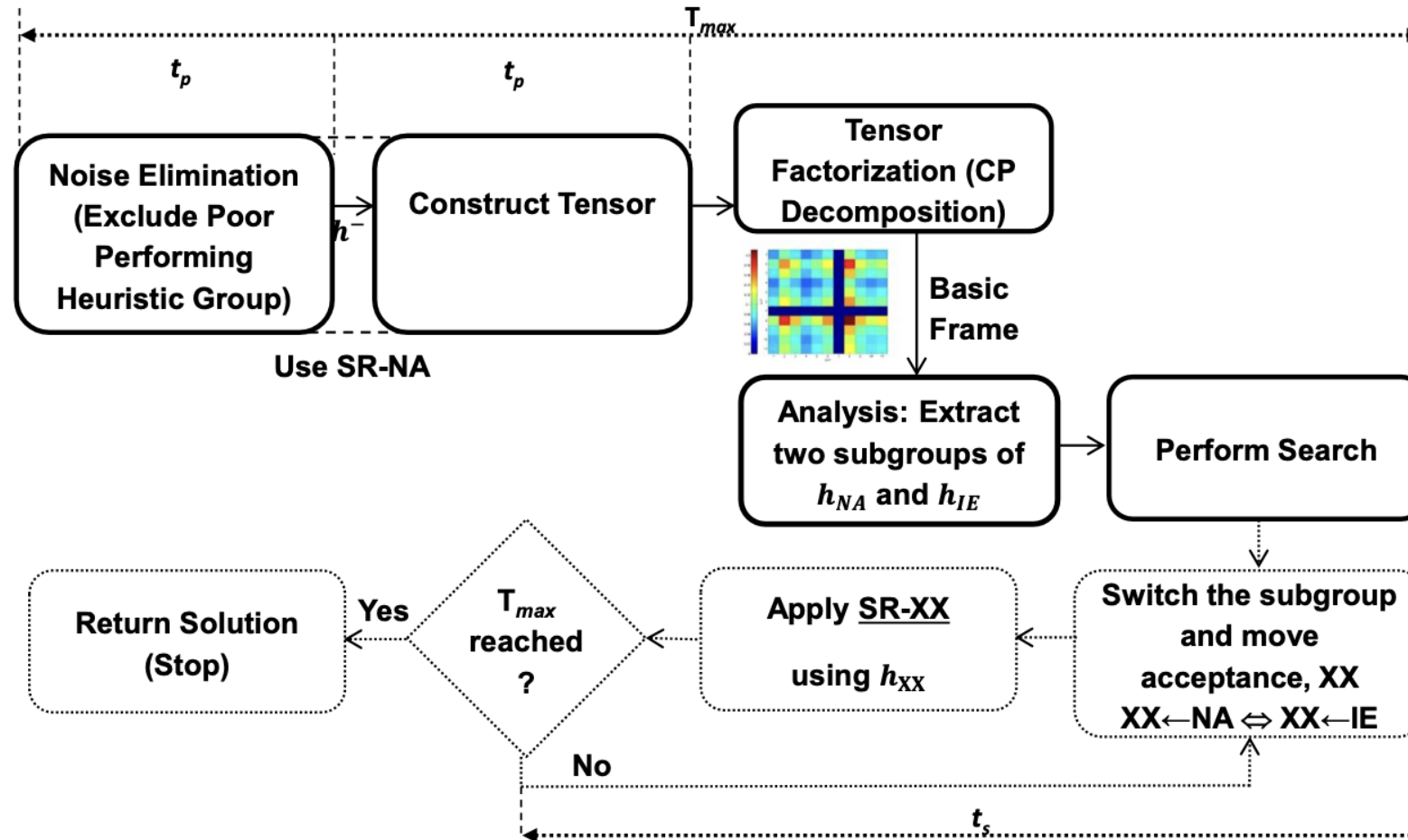


Proposed Approach - Ideas

- The notation $h_{IE} \cup h_{NA}$ refers to the union of sets of low-level heuristics associated with each acceptance method, with conditions ensuring they are distinct (i.e., $h_{IE} \cap h_{NA} = \emptyset$).



Proposed Approach – TeBHA-HH



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Proposed Approach – TeBHA-HH

Tensor Based Hybrid Acceptance Hyper-heuristic (TeBHA-HH) which consists of five consecutive phases: (i) noise elimination (ii) tensor construction, (iii) tensor factorization, (iv) tensor analysis, and (v) hybrid acceptance as illustrated in Figure 2. The noise elimination filters out a group of low level heuristics from h and then a tensor is constructed using the remaining set of low level heuristics, denoted as h^- . After tensor factorization, sub-data describing the latent relation between low level heuristics is extracted. This information is used to divide the low level heuristics into two partitions: h_{NA} , h_{IE} . Each partition is then associated with a move acceptance method, that is naive move acceptance with $\alpha = 0.5$ (NA) or improving and equal moves (IE) respectively. This is equiv-

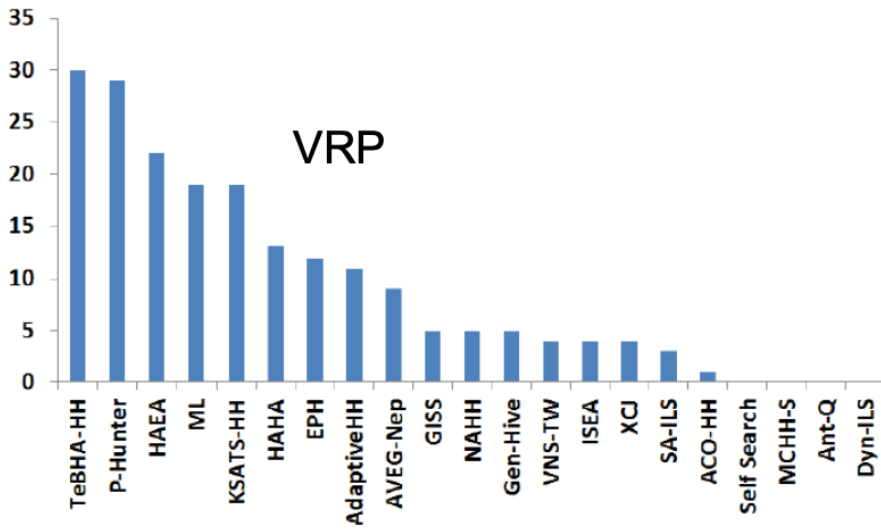
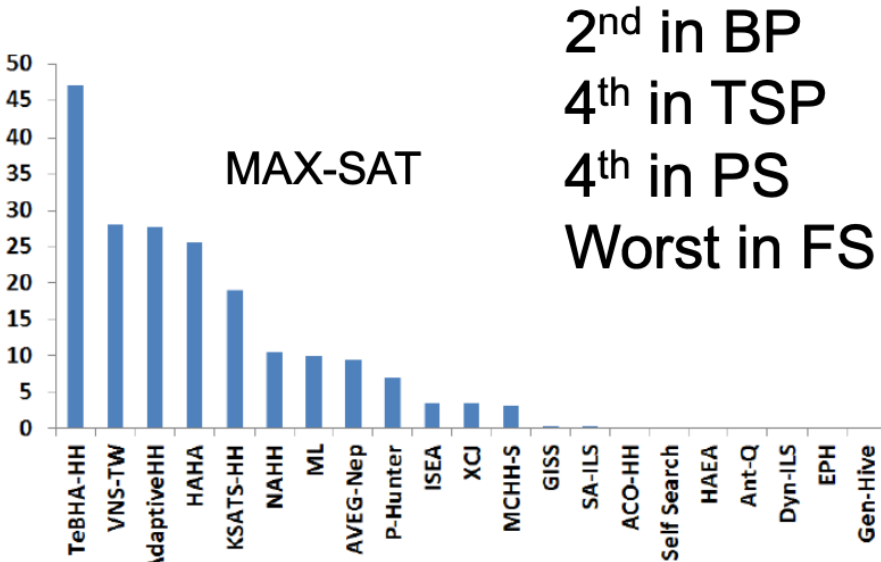


Proposed Approach – TeBHA-HH

alent to employing two selection hyper-heuristics, Simple Random-Naive move acceptance (SR-NA) and Simple Random-Improving and Equal (SR-IE). Each selection hyper-heuristic is invoked in a round-robin fashion for a fixed duration of time (t_s) using only the low level heuristics associated with the move acceptance component of the hyper-heuristic at work (h_{NA} and h_{IE} , respectively) until the overall time limit (T_{max}) is reached. This whole process is repeated at each run while solving a given problem instance. All the problems dealt with in this study are minimizing problems. A detailed description of each phase is



Results

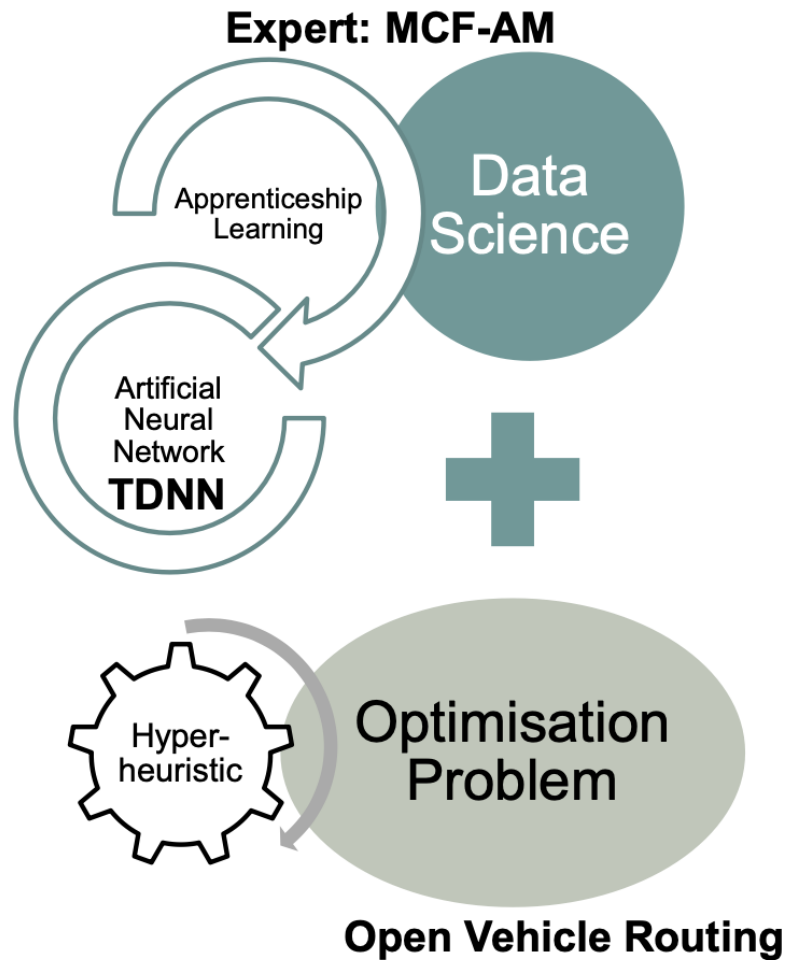


Rank	Name	Score
1	AdaptiveHH	162.83
2	TeBHA-HH	148.85
3	VNS-TW	118.83
4	ML	117.50
5	P-Hunter	84.75
6	EPH	83.25
7	NAHH	68.50
8	HAHA	65.58
9	ISEA	62.50
10	KSATS-HH	52

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



A Single Objective Hyper²-heuristic – **Apprenticeship Learning**



<http://www.calinon.ch/images/hoap2-xsens01.jpg>



Automated generation of a new hyper-heuristic by observing an expert hyper-heuristic in operation

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Apprenticeship learning

- **Apprenticeship learning** is a machine learning paradigm where an agent learns to perform a task by observing an expert.
- The core idea is to enable the learner to mimic the expert's behavior rather than explicitly programming rules or algorithms for the task.



Results on Open VRP

	C1	C2	C3	C4	C5	C6	C7
(1)	5432.4	10727.2	8728.9	12934.1	18054.1	5551.6	10671.2
(2)	5560.5	10944.4	8925.6	13216.4	18168.5	5543.7	10937.9
$W(2)$	<	<	<	<	<	$\geq$	<
(3)	5534.3	10856.6	8838.5	13091.2	17502.2	5524.4	10860.8
$W(3)$	<	<	<	<	>	$\geq$	<
	C8	C9	C10	C11	C12	C13	C14
	8945.4	14094.3	18131.6	7976.8	10699.9	11579.9	10963.9
	8925.4	14229.3	18578.7	8279.1	10901.8	11539.2	10935.3
	$\geq$	<	<	<	<	$\geq$	$\geq$
	8958.4	13135.4	17511.1	8068.7	10827.6	11417.2	10839.9
	<	>	>	<	<	$\geq$	$\geq$

(1) vs (2)

$\leq$: 10

$>$: 0

$\geq$: 4

(3) vs (2)

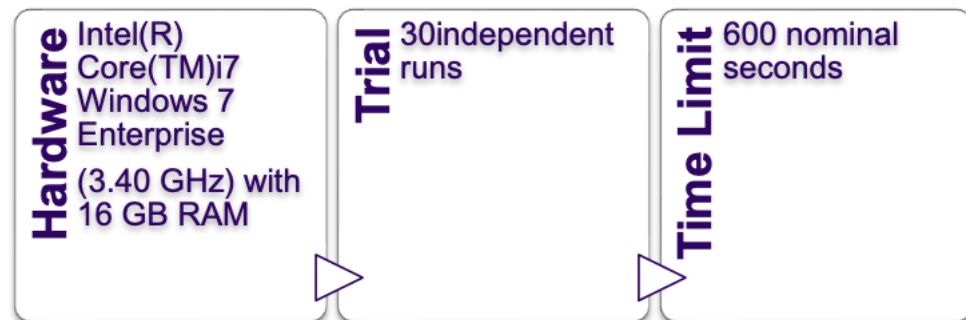
$\leq$: 8

$>$: 3

$\geq$: 3

R. Tyasnurita, E. Özcan and R. John, Learning Heuristic Selection using a Time Delay Neural Network for Open Vehicle Routing, Proc. of the 2017 IEEE Congress on Evolutionary Computation (CEC), pp. 1474-1481.

- (1) TDNN-ALHH (objective values and distance)
- (2) EXPERT (MCF-AM)
- (3) TDNN-ALHH (objective values)



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Summary I

- Hyper-heuristic research originated from a job shop scheduling application and has been rapidly growing since then.
- Generation hyper-heuristics are commonly used in the area
 - Train and test fashion
 - Does the selected subset of training instances is sufficiently representative of the test set?
 - Training is time-consuming (delta/incremental evaluation, surrogate functions)
 - The **evolved heuristics** might not be easy to interpret, yet they can outperform human designed heuristics

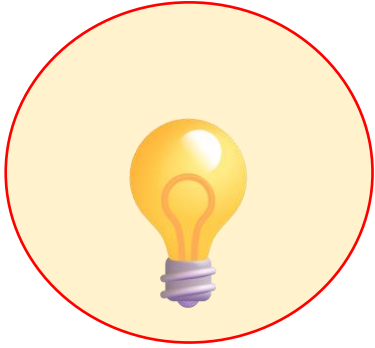


Summary II

- There is empirical evidence that **machine learning/analytics/ data science help to improve the hyper-heuristic search process**
 - Problem features vs solution/state features
 - Offline versus online learning – Life long learning
- There is still a lack of benchmarks
- **Automated design of search methodologies** is extremely challenging
 - Addressed in almost complete absence of a mathematical and theoretical understanding



Summary



1. Policy Matrix Evolution for Generation of Heuristics
2. Data Science (Machine Learning)
Improved Hyper-heuristic Optimisation



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



University of
Nottingham

UK | CHINA | MALAYSIA

Questions



University of
Nottingham

UK | CHINA | MALAYSIA

NEXT:

Knowledge & Reasoning